Vehicle Studio Image API

Integration guide for developers | API v1 | 19 October 2026

1. Overview

The API turns vehicle photos into studio-style PNG images on a transparent background (see the Technical Overview for what the processing does). It is a JSON REST API over HTTPS. Processing is **asynchronous**: one **job** per vehicle holds all of its photos; you upload the photos, submit the job, and receive the results by **webhook** or by polling.

Step	Call	Purpose
1	`POST /v1/jobs`	Create a job for one vehicle; get one upload link per photo
2	`PUT {upload_url}`	Upload each photo directly to secure storage (or give us source URLs)
3	`POST /v1/jobs/{job_id}/submit`	Start processing
4	Webhook `job.completed`	We call your endpoint when the job finishes (recommended)
4b	`GET /v1/jobs/{job_id}`	Alternatively, poll for status
5	`GET {result.url}`	Download each finished PNG

A machine-readable OpenAPI 3.1 description of this API is supplied with this guide (`openapi.yaml`) for Postman, Swagger UI or client code generation.

2. Environments and authentication

Environment	Base URL	API key prefix	Notes
Production	`https://{base_url}/v1`	`ax_live_`	Billed per processed image
Sandbox	`https://{sandbox_url}/v1`	`ax_test_`	Same API, no charge, for integration testing

Base URLs and API keys are issued at onboarding. Send the key on every request:

```
Authorization: Bearer ax_live_xxxxxxxxxxxxxxxxxxxxxxxx
```

- Keep API keys on your servers only. Never put them in mobile apps or browser code.
- Keys can be rotated on request; old keys stay valid for 7 days after rotation.
- All traffic must use HTTPS (TLS 1.2+). Requests and responses are UTF-8 JSON unless stated otherwise.

3. Quick start (curl)

1. Create a job for one vehicle with its photos:

```
curl -X POST https://{base_url}/v1/jobs \
  -H "Authorization: Bearer $AXION_API_KEY" \
  -H "Content-Type: application/json" \
  -H "Idempotency-Key: STOCK-48213-2026-10-19" \
  -d '{
    "vehicle_ref": "STOCK-48213",
    "tier": "studio",
    "webhook_url": "https://client.example.com/hooks/axion",
    "images": [
      {"image_ref": "01_front", "view": "exterior",
       "filename": "01_front.jpg", "content_type": "image/jpeg"},
      {"image_ref": "13_dashboard", "view": "interior",
       "filename": "13_dashboard.jpg", "content_type": "image/jpeg"}
    ]
  }'
```

The response (`201 Created`) contains a `job_id` and, per image, an `upload_url` and the `upload_headers` to send:

```
{
  "job_id": "job_01J9ZKQ4M7",
  "status": "awaiting_upload",
  "tier": "studio",
  "upload_expires_at": "2026-10-19T11:05:00Z",
  "images": [
    { "image_id": "img_7Q2M", "image_ref": "01_front", "view": "exterior",
      "status": "awaiting_upload",
      "upload_url": "https://storage.../01_front.jpg?signature=...",
      "upload_headers": { "Content-Type": "image/jpeg" } },
    ...
  ]
}
```

2. Upload each photo with a plain HTTP PUT of the file bytes to its `upload_url`:

```
curl -X PUT "<upload_url>" -H "Content-Type: image/jpeg" --data-binary @01_front.jpg
```

3. Submit the job once every photo is uploaded:

```
curl -X POST https://{base_url}/v1/jobs/job_01J9ZKQ4M7/submit \
  -H "Authorization: Bearer $AXION_API_KEY"
```

4. Receive the result by webhook (section 6) or poll every 10–15 seconds:

```
curl https://{base_url}/v1/jobs/job_01J9ZKQ4M7 -H "Authorization: Bearer $AXION_API_KEY"
```

5. Download each image's `result.url` (a PNG with a transparent background) before `result.url_expires_at`. You can fetch a fresh link at any time with `GET /v1/jobs/{job_id}` while the job's results are retained.

4. Endpoints

Method and path	Description	Success
`POST /v1/jobs`	Create a job and get upload links. Supports `Idempotency-Key`.	201 Job
`POST /v1/jobs/{job_id}/submit`	Start processing. All images must be uploaded (or have a `source_url`).	202 Job
`GET /v1/jobs/{job_id}`	Job status, per-image status and result links.	200 Job
`GET /v1/jobs`	List jobs, newest first. Filters: `status`, `vehicle_ref`; paging: `limit` (max 100), `cursor`.	200 list
`DELETE /v1/jobs/{job_id}`	Cancel a job before processing starts, or delete a finished job's stored photos and results early.	204
`POST /v1/jobs/{job_id}/images/{image_id}/reprocess`	Re-process one image of a finished job, for example if a result does not match the photo.	202 Job
`GET /v1/usage?from=&to=`	Processed-image counts per tier for a date range, for invoice reconciliation.	200 Usage

Create-job request fields

Field	Type	Required	Description
`tier`	string	yes	`standard` (original lighting), `studio` (neutral studio lighting) or `studio_plus` (studio, higher detail)
`images`	array (1–40)	yes	One entry per photo of the vehicle
`images[].image_ref`	string	yes	Your identifier, unique within the job; echoed in results
`images[].view`	string	yes	`exterior` or `interior` (cabin / open-door shots)
`images[].content_type`	string	no	`image/jpeg`, `image/png`, `image/webp` or `image/heic`
`images[].source_url`	URL	no	HTTPS link we download instead of you uploading (public or pre-signed, valid for at least 1 hour)
`vehicle_ref`	string	no	Your stock number, VIN or listing ID
`webhook_url`	URL	no	HTTPS endpoint for the job-finished event
`metadata`	object	no	Up to 20 string key/value pairs, returned unchanged

5. Job and image status

Job status	Meaning
`awaiting_upload`	Created; waiting for photos and submit. Upload links expire after 60 minutes.
`queued` / `processing`	Submitted; images are being processed in parallel.
`completed`	All images succeeded.
`completed_with_errors`	Some images failed; the others have results. See each image's `error`.
`failed`	No image could be processed.
`cancelled` / `expired`	Cancelled by you, or never submitted within 24 hours.

Image status: `awaiting_upload` → `uploaded` → `queued` → `processing` → `succeeded` or `failed`. Typical processing time is 20–25 seconds per image; a 12-photo vehicle typically completes in about 1.5–2 minutes.

Example of a finished image in `GET /v1/jobs/{job_id}`:

```
{ "image_id": "img_7Q2M", "image_ref": "01_front", "view": "exterior",
  "status": "succeeded", "processing_ms": 21480,
  "result": { "url": "https://storage.../01_front.png?signature=...",
              "url_expires_at": "2026-10-19T12:10:00Z",
              "content_type": "image/png", "width": 1536, "height": 1024, "bytes": 2483112 },
  "error": null }
```

6. Webhooks

If you set `webhook_url`, we send one `POST` when the job finishes, with event type `job.completed`, `job.completed_with_errors` or `job.failed`. The body is the full job (same as `GET /v1/jobs/{job_id}`).

```
POST /hooks/axion
Content-Type: application/json
X-Axion-Event-Id: evt_01J9ZM2B8T
X-Axion-Signature: t=1760870000,v1=5b1f0c...e9a2

{ "event_id": "evt_01J9ZM2B8T", "type": "job.completed",
  "created_at": "2026-10-19T11:02:41Z",
  "data": { "job": { "job_id": "job_01J9ZKQ4M7", "status": "completed",
                     "images": [ ... ] } } }
```

- **Verify the signature** before trusting the payload: compute HMAC-SHA256 of `<t>.<raw body>` with your webhook secret (issued at onboarding) and compare it to `v1`. Reject events whose `t` is more than 5 minutes old.
- Respond with any 2xx within 10 seconds, then process asynchronously. Failed deliveries are retried with increasing delays for up to 24 hours.
- Events can arrive more than once: de-duplicate on `event_id`.

Signature check: Node.js

```
import crypto from "node:crypto";

export function verifyAxionSignature(rawBody, header, secret) {
  const parts = Object.fromEntries(header.split(",").map((kv) => kv.split("=")));
  const signed = `${parts.t}.${rawBody}`;
  const expected = crypto.createHmac("sha256", secret).update(signed).digest("hex");
  const fresh = Math.abs(Date.now() / 1000 - Number(parts.t)) < 300;
  return fresh &&
    crypto.timingSafeEqual(Buffer.from(expected), Buffer.from(parts.v1));
}
```

Signature check: Python

```
import hashlib, hmac, time

def verify_axion_signature(raw_body: bytes, header: str, secret: str) -> bool:
    parts = dict(kv.split("=", 1) for kv in header.split(","))
    signed = parts["t"].encode() + b"." + raw_body
    expected = hmac.new(secret.encode(), signed, hashlib.sha256).hexdigest()
    fresh = abs(time.time() - int(parts["t"])) < 300
    return fresh and hmac.compare_digest(expected, parts["v1"])
```

7. Results and retention

- Each result is a PNG (RGBA, 8-bit per channel) with a transparent background, about 1536 × 1024 px for landscape photos.
- Result links are pre-signed and valid for 60 minutes; request the job again for fresh links.

- Uploaded photos and results are kept for 30 days after the job finishes, then deleted automatically. Use `DELETE /v1/jobs/{job_id}` to delete them earlier. Store the PNGs on your side.

8. Errors

Errors use standard HTTP status codes and a JSON body:

```
{ "error": { "code": "invalid_request",
             "message": "images[3].view must be 'exterior' or 'interior'",
             "request_id": "req_8KQ2N1", "retry_after_seconds": null } }
```

HTTP	code	When	Retry?
400	`invalid_request`	Missing or invalid field	No, fix the request
401	`unauthorized`	Missing or invalid API key	No
404	`not_found`	Unknown job or image	No
409	`invalid_state`	Action not allowed in the job's current status (for example, submit twice)	No
413	`file_too_large`	Photo larger than 25 MB	No
415	`unsupported_media_type`	File type not supported	No
422	`images_missing`, `image_unreadable`	Submit before all uploads finished, or a photo cannot be decoded	After fixing
429	`rate_limited`	Too many requests; see `Retry-After`	Yes, after the delay
500 / 503	`internal_error`, `unavailable`	Temporary problem on our side	Yes, with backoff

Per-image errors appear in `images[].error` with the same `code`/`message` shape. Always log `request_id` when contacting support.

9. Limits

Limit	Value
Photos per job	1–40 (one vehicle per job)
Photo file size	Up to 25 MB; minimum 1024 px on the long side
API requests	60 per minute per API key (higher limits on request)
Jobs processing at the same time	10 per API key; further jobs wait in the queue
Upload link validity	60 minutes; unsubmitted jobs expire after 24 hours
Idempotency-Key	Remembered for 24 hours; the same key returns the original job

10. Complete example: Node.js

```
const API = "https://{base_url}/v1";
const headers = {
  Authorization: `Bearer ${process.env.AXION_API_KEY}`,
  "Content-Type": "application/json",
};

async function processVehicle(vehicleRef, photos /* [{ ref, view, path, type }] */) {
  // 1. create the job
  const job = await fetch(`${API}/jobs`, {
    method: "POST",
    headers: { ...headers, "Idempotency-Key": `${vehicleRef}-v1` },
    body: JSON.stringify({
      vehicle_ref: vehicleRef,
      tier: "studio",
      webhook_url: "https://client.example.com/hooks/axion",
      images: photos.map((p) => ({ image_ref: p.ref, view: p.view, content_type: p.type })),
    }),
  }).then((r) => r.json());

  // 2. upload every photo to its pre-signed link
  const fs = await import("node:fs/promises");
  await Promise.all(job.images.map(async (img) => {
    const photo = photos.find((p) => p.ref === img.image_ref);
    const body = await fs.readFile(photo.path);
    await fetch(img.upload_url, { method: "PUT", headers: img.upload_headers, body });
  }));

  // 3. submit; results arrive at the webhook (or poll GET /jobs/{job_id})
  await fetch(`${API}/jobs/${job.job_id}/submit`, { method: "POST", headers });
  return job.job_id;
}
```

11. Versioning and support

- The version is part of the path (`/v1`). New optional fields, endpoints and event types may be added to v1 at any time; ignore fields you do not recognise. Breaking changes only come in a new version, announced at least 90 days ahead.
- Test your integration in the sandbox first; sandbox results are watermarked and not billed.
- For integration help, contact your account manager with the `request_id` of the call in question.